

1. Configuración del Proyecto (Maven y pom.xml)

¿Para qué sirve? Maven es un gestor de dependencias. En proyectos profesionales, no descargamos archivos .jar manualmente porque es difícil mantenerlos actualizados.

- **El archivo pom.xml:** Es el "cerebro" de la configuración. Al añadir la dependencia mysql-connector-j, le decimos a Java que descargue el controlador oficial para hablar con MySQL. Sin esto, Java no sabría cómo interpretar los comandos que enviamos hacia la base de datos.
-

2. Clase Conexion.java (El Puente)

Aquí explicamos el código de la conexión:

- **import java.sql.*:** Importamos las herramientas básicas de SQL.
 - **DriverManager.getConnection(url, user, password):** Este es el método más importante. La url es la dirección de tu base de datos (localhost:3306). Es como marcar un número de teléfono; si los datos coinciden, se abre el canal de comunicación.
 - **Bloque try-catch:** Lo usamos porque la conexión a una base de datos puede fallar (si el servidor está apagado o la contraseña es incorrecta). En lugar de que el programa "trueque" y se cierre, el catch atrapa el error y nos avisa qué pasó.
-

3. Clase Main.java (Lógica del Juego)

El Ciclo do-while y switch-case

- **¿Por qué lo usamos?:** Un juego necesita mantenerse "vivo" hasta que el usuario decida salir. El do-while asegura que el menú se imprima al menos una vez y se repita infinitamente mientras la opción no sea 5.
- **leer.nextLine():** Después de un nextInt(), el programa deja un "salto de línea" fantasma en la memoria. Usamos esto para limpiar el teclado y que no se salte las preguntas de texto.

El uso de PreparedStatement (Seguridad)

En todas las funciones (registrar, actualizar, eliminar), usamos PreparedStatement en lugar de un Statement normal.

- **¿Para qué sirve?:** Sirve para usar los signos de interrogación ?. Esto separa el comando SQL de los datos que escribe el usuario. Si no lo usáramos, un usuario malintencionado podría borrar toda tu base de datos (ataque de Inyección SQL). Además, permite que el código sea más ordenado.

Métodos CRUD (Explicación técnica):

1. **registrarPokemon (Create):**

- Usa `ps.executeUpdate()`. Este método se usa para comandos que modifican la tabla (INSERT, UPDATE, DELETE). Devuelve cuántas filas se afectaron.
 - 2. **actualizarRegistros (Update):**
 - Aquí implementamos una sub-lógica. El programa decide si ataca la tabla pokemones o mochilas. Es importante usar el **ID** en el WHERE del comando SQL; sin el ID, actualizaríamos a todos los pokemones al mismo tiempo.
 - 3. **eliminarPokemon (Delete):**
 - Simplemente borra un registro. Es la acción más delicada y por eso se filtra estrictamente por el ID único que genera MySQL.
 - 4. **mostrarRegistros (Read):**
 - **ResultSet rs:** Es un objeto que actúa como un "puntero" o cursor sobre los resultados que arroja la base de datos.
 - **while (rs.next()):** Este bucle le dice a Java: "Mientras haya una fila más adelante, muévela y léela".
 - **rs.getInt y rs.getString:** Extraen el valor de una columna específica del renglón donde está parado el cursor actualmente.
-

4. Base de Datos (MySQL)

- **AUTO_INCREMENT:** Lo usamos en el ID para que nosotros no tengamos que inventar números. MySQL le asigna el 1, luego el 2, y así sucesivamente, asegurando que ningún registro se confunda con otro.
 - **PRIMARY KEY:** Es el identificador único. Es como el CURP de los datos; no puede haber dos iguales.
 - **Datos precargados (Mochila):** El hecho de insertar las pociones directamente en SQL (y no en Java) simula un juego real donde el inventario ya existe cuando el jugador inicia su partida.
-

1. Clase Conexion.java (Gestión de Base de Datos)

Esta clase tiene una única responsabilidad: establecer la comunicación entre nuestra aplicación Java y el servidor MySQL.

- **import java.sql.*;** Se importan las librerías nativas de Java (Connection, DriverManager, SQLException) necesarias para manejar bases de datos relacionales (JDBC).

- **public static Connection getConnection():** Es un método de tipo Connection y estático (static). Al ser estático, nos permite llamarlo desde cualquier otra parte del programa (como en el Main) sin tener que crear un objeto de tipo Conexion cada vez.
- **Definición de credenciales (url, user, password):**
 - La url (jdbc:mysql://localhost:3306/juego_pokemon) le indica a Java el protocolo (jdbc), el motor (mysql), el servidor (localhost), el puerto por defecto (3306) y el nombre exacto de la base de datos a la que vamos a apuntar.
- **DriverManager.getConnection(...):** Es la línea que físicamente intenta "abrir la puerta" hacia la base de datos usando las credenciales proporcionadas.
- **Bloque try-catch:** Conectarse a una base de datos es un proceso externo que puede fallar (por ejemplo, si el servidor XAMPP/MySQL está apagado). El bloque try intenta la conexión, y si algo sale mal, el catch (SQLException e) atrapa el error y lo muestra en pantalla, evitando que la aplicación colapse bruscamente.

2. Clase Main.java (Lógica y Menú Interactivo)

Esta es la clase principal que ejecuta el programa. Contiene el menú interactivo y los métodos CRUD (Crear, Leer, Actualizar, Borrar).

El Menú Principal (public static void main)

- **Scanner leer = new Scanner(System.in);:** Instanciamos un objeto Scanner para capturar la entrada de datos por teclado del usuario.
- **Ciclo do-while:** Se utiliza esta estructura de control iterativa porque necesitamos que el menú se muestre en pantalla **al menos una vez**. El ciclo se repetirá infinitamente mostrando las opciones hasta que el usuario digite el número 5.
- **switch (opcion):** Estructura de control condicional que evalúa la opción elegida por el usuario y dispara el método correspondiente (registrarPokemon, eliminarPokemon, etc.).
- **leer.nextLine();:** Esta línea se coloca después de un leer.nextInt() para "limpiar el buffer" del teclado. Evita que el programa se salte las siguientes lecturas de texto por culpa de un salto de línea residual en la memoria.

Método registrarPokemon (INSERT)

- **Captura de datos:** Se piden los valores al usuario y se guardan en variables de tipo String (nombre, tipo).
- **PreparedStatement ps = cxn.prepareStatement(sql);:** En lugar de enviar un texto SQL estático, usamos sentencias preparadas. La consulta se escribe con signos de interrogación (VALUES (?, ?)).
- **ps.setString(1, nombre);:** Aquí se sustituyen los signos de interrogación por las variables capturadas. Esto es una buena práctica de programación que **previene ataques de Inyección SQL**, haciendo el sistema mucho más seguro.

- **ps.executeUpdate();**: Es el comando que envía la instrucción de inserción a MySQL y aplica los cambios físicamente en la tabla.

Método actualizarRegistros (UPDATE)

- **Lógica de sub-menú**: Usamos un if-else para separar la actualización. Si el usuario elige 1, atacamos la tabla pokemones; si elige 2, atacamos la tabla mochilas.
- **Filtrado por WHERE id = ?**: Para actualizar, es obligatorio solicitar el id del registro. Si se omitiera la cláusula WHERE en la sentencia UPDATE, todos los registros de la tabla cambiarían al mismo valor, arruinando la integridad de la base de datos. Se actualizan los datos usando nuevamente ps.executeUpdate().

Método eliminarPokemon (DELETE)

- **Instrucción DELETE FROM**: Se captura el id del Pokémon que el usuario desea liberar.
- Al igual que en la actualización, se utiliza un PreparedStatement para inyectar el ID de forma segura en la consulta DELETE FROM pokemones WHERE id = ?. Posteriormente, se ejecuta el cambio en la base de datos.

Método mostrarRegistros (SELECT)

- **ResultSet rs = ps.executeQuery();**: A diferencia de los métodos anteriores que *modifican* datos (y usan executeUpdate), aquí queremos *leer* datos, por lo que usamos executeQuery. Lo que nos devuelve MySQL se guarda en un objeto especial llamado ResultSet. El ResultSet actúa como una tabla virtual en la memoria de Java.
- **Ciclo while (rs.next())**: El ResultSet funciona con un cursor. El método .next() mueve el cursor fila por fila. El ciclo se repite mientras sigan existiendo filas por leer en la tabla.
- **Extracción de datos (rs.getInt, rs.getString)**: Estando posicionados en una fila específica, usamos estos métodos indicando el nombre exacto de la columna en la base de datos para extraer su valor y concatenarlo en un System.out.println() para mostrarlo ordenado en la consola.